Lecture 08: memory

Goals

- build a circuit that can store more than a single bit
- build a *memory* that can store separately addressable chunks of data
- define *nibble*, *byte*, and *octet*
- define *address*, *addressability*, and *address space* and relate them mathematically
- use splitters to cause wires to carry more than one bit at a time

on Monday, we saw the RS latch, the gated D-latch, and the D flip-flop

these storage elements are nice, but they only let us store a single bit

that's kinda limiting

how many programs you've written have used values that were a single bit?

my guess is you've used booleans fairly frequently, actually, as invariants in while loops, conditions in if-s variables

but you also used types that could take on more than two values

integers

character strings

today we're going to talk about how to get from storing single bits to storing multi-bit values

so we want to store a value that requires more than 1 bit

there's a pretty obvious solution to this

group together 4 D-flip-flops and we get a thing that can store a 4-bit value

we'll call this piece of hardware that stores a fixed-size value a *register*

(I don't know the origin of the term)

4_bit_register.circ

you've heard of a "byte"

it's an 8-bit value (though back in the day this wasn't always the case: there exist old papers where a byt

a less ambiguous term for an 8-bit value is "octet"

a 4-bit value is sometimes called a "nibble"

this is, admittedly, a contrived example: 4 bits isn't much better than 1

hopefully, though, it's clear how we can easily extrapolate this to larger quantities

and we'll do that eventually, but this small example is useful for now

this circuit is still a bit problematic: if we imagine extrapolating it to store, say, 32-bit values, we're going t wires, making our circuits utterly inscrutable

fortunately, Logisim allows us to gather together a bunch of wires into a bundle (for which it does not hav you go)

the circuit element that allows us to do this is the *splitter* (available under the `Wires` menu)

the splitter is used to both a) combine together a bunch of individual wires into a single bundle and b) se constintuent individual wires

in the spliter's properties, the "Bit Width In" property specifies how many wires are hiding inside the bunc carries 4 bits, I would set the "Bit Width In" property to 4

the "Fan Out" property controls how many individual wires I want on the other end of the splitter; for exar coming in and I want separate output wires for each of those 4, I would set the "Fan Out" property to 4

if, however, I have a 4-wire bundle coming in and I would like to separate it into two 2-bit wires, I would s the "Bit n" properties lower down to control which bit within the bundle goes to which output on the splitte

here is our previous 4-bit register modified to use splitters: 4_bit_register_with_splitters.circ

the input and output pins also have a "Bit Width" of 4

you can change individual bits of the input by poking them with the Poke tool

taking things in a different direction, we might want to store more than a single 4-bit value

I'm guessing most (all?) of the remotely interesting/useful programs you've written have used more than

therefore, it would be really handy to have hardware that can store more than one thing at once

not only do we want to store multiple things, we want to be able to refer to the things individually

such a piece of hardware is called a *memory*

before thinking about laying down circuitry, though, we ought to think about what kind of behavior we wa

if we think about how we've used variables, it all boils down to two operations: we use its value ("read") c ("write")

we also want the ability to store several values (variables) simultaneously and therefore we need some v one we're referring to in any particular operation

it will probably not surprise you to learn that we will use registers to store these values

if we want to store three variables, we would need three separate registers

to write a new value, we need to specify that new value and also identify the specific register we want tha

to read a value, we need to identify the specific register whose value we want

for the sake of example, let's get concrete and decide that we want to store three separate 4-bit values

to refer to them, the easiest way is to just assign them arbitrary numbers: value 0, value 1, value 2

if we've got three registers, how many bits do we need to identify a register?

2, because with 2 bits we can specify 2^2 = 4 different values, which gives us a unique sequence of bits

this idea of identifying the item by its location is common in memory, and we refer to the location as the *a*

so we would say "the nibble at address 0" or "the nibble at address 2"

we have now fully specified the inputs to this circuit we have in mind:

- write_value: 4 bits
- write_addr: 2 bits
- clk: 1 bit
- read_addr: 2 bits

and the output: - read_value: 4 bits

I didn't explicitly mention the clock above; it's necessary because the flip-flops inside the registers inside when to store a new value

the reading functionality will be unrelated to the clock: if we change the value of read_addr, we should se register show up on the read_value output shortly thereafter

again deferring our desire to begin laying down circuitry, it's helpful to think even more concretely about t aiming for

the level of detail I have in mind is "if we supply inputs X, Y, and Z, then the output should be ABC"

if you're thinking this is the beginning of a test plan, you're right

it also helps sharpen our conception of the behavior so that we don't go off on a wild tangent when we st

if I set write_addr to 01 and write_value to 0101 and tick the clock, the register at address 01 should stor

if I then set write_addr to 10 and write_value to 1010 and tick the clock, the register at address 10 shoulc

then, if I set read_addr to 01, I should see 0101 on the read_value output

and if I set read_addr to 10, I should see 1010 on the read_value output

we can now start to think about how to make this stuff happen

let's start with the read operation

we've got three registers, and we want to be able to choose one whose value should appear on the outp

whenever you think about choice, you need to think "multiplexer"

the write operation is a bit trickier

the incoming data value could potentially end up in any one of the three registers, so we need to connec input of all three registers

but we only want a single one of those registers to actually store that new value

this is where a decoder is handy: it has N outputs and lets us pick exactly one of those to be 1, while the

so if we use the write_addr input to tell the decoder which of its outputs to be 1, we can use that to activa

here it is: 4_by_3_memory_first_try.circ

some other vocabulary now appears!

there are a total of three different addresses: three different places where we can store data

therefore, the size of the *address space* of this memory is 3

the address space of a memory is the number of addresses it supports

often, "address space" is described, not by the number of addresses, but by the number of bits in an adc related to the number of addresses)

if I have a memory that contains 256 different places to store a piece of data, it requires 256 addresses, an address

I could say the address space has size 256 or I could say it has an "8-bit address space" (because the a

each item in our memory is 4 bits: one nibble

this is the *addressability*: the amount of data stored at each address

implicit here is that *every* address stores the same amount of data; this is pretty reliably true

modern machines are almost universally *byte-addressable*, meaning that each address refers to one byt

if we've got a 32-bit address space with byte-addressable memory, we've got 2^32 * 8 bits of memory

there is a significant problem with the circuit above, though

if clk is high and we change the value of the write address, the stored value changes

this behavior is INCORRECT given our belief/desire that a new value is saved only on the rising edge of

(the reasons for this will become apparent in the coming weeks; for now, please just accept this is import

the immediate cause of this particular problem is that the "clk" signal on each of the flip-flops is determin
"clk" input of the entire circuit

specifically, it depends on the *combination* of the "clk" input and the decoder's output

so when the decoder's output changes, one row of flip-flops may see a rising edge on each individual flip
main clk input didn't change

this is because we've put a gate (specifically, an and-gate) between the clk input and where it is being us

doing so is referred to as "gating the clock" and is considered a major no-no in chip design

therefore, to solve this problem, we're going to take the built-in D-flip-flop circuit component and wrap it i
its behavior to do what we want

here it is: d_flip_flop_with_enable.circ

a couple things to notice before diving in:

- first, the clock signal goes directly to the flip-flop sub-component
- secondly, the currently-stored bit is an input to the combinational logic on the left

that combinational logic on the left answers the question, "if the clock ticks, what bit should be stored ne

if the "en"able input is 0, the bit stored should be the same as the bit already stored (ie, it shouldn't chan
—so we just write the old bit again

if, however, "en" is set to 1, we need to write the new, incoming bit, whether it is a zero or a one

the weird, perhaps counterintuitive thing, is that the flip-flop *always* stores a value on the rising edge of t
logic on the left makes sure that value being stored is correct based on the "en" and "D" inputs

we just replace the old, basic, flip-flops-with-no-enable with our new ones and we're good to go:

4_by_3_memory.circ

Definitions

The following definitions introduced in this lecture are fair-game for future quizzes. You will be expected t
provided in these lecture notes.

- memory
- ~~byte~~
- ~~nibble~~
- ~~octet~~
- address
- address space
- addressability

Last modified: 09/30/2026 14:56:55